



Deploy Backend with Kubernetes



Natasha Ong

```
[ec2-user@ip-172-31-29-112 manifests]$ kubectl apply -f flask-deployment.yaml
kubectl apply -f flask-service.yaml
deployment.apps/nextwork-flask-backend created
service/nextwork-flask-backend created
[ec2-user@ip-172-31-29-112 manifests]$
```



Introducing Today's Project!

In this project, we will deploy the backend of an application with Kubernetes! This project will start from scratch (i.e. launching an EC2 instance) and go all the way to having a live backend deployed.

Tools and concepts

We used Kubernetes, ECR, kubectl, Docker, eksctl, GitHub to deploy the backend of an application. Key concepts include applying manifest files, building container images and pushing them to a container registry

Project reflection

This project took us approximately two hours (including demo time). The most challenging part was creating the manifest files. Our favourite part was verifying the deployment in the EKS console.



Project Set Up

Kubernetes cluster

To set up today's project, I launched a Kubernetes cluster. The cluster's role in this deployment is to be the control centre for using Kubernetes and orchestrating all the containers that will be running the application we want to deploy.

Backend code

We retrieved backend code by cloning a repository in GitHub so we have a local copy of our team member's work. Pulling code is essential because we need to access it to build a container image of our application so that Kubernetes will deploy it.

Container image

Once we cloned the backend code, we built a container image because Kubernetes needs our application to be packaged in an image to deploy it. Without an image, it would be difficult for Kubernetes to deploy our application consistently across a cluster.

I also pushed the image to a container registry, which is a storage space for container images in the cloud. ECR facilitates scaling for my deployment because Kubernetes can easily use whichever image is tagged latest for each container it creates.



Manifest files

Kubernetes manifests are the instructions/manuals on how we'd like Kubernetes to deploy, expose and run our containerized application. Manifests are helpful because it allows Kubernetes to repeat a deployment using the same instructions.

A deployment manifest manages the creation/setting up of a Deployment resource that's responsible for creating containers/rolling out updates across our EKS cluster. The container image URL in my Deployment manifest tells Kubernetes where my image is

A Service exposes a deployed application to users/traffic. Our Service manifest sets up a Service resource that exposes our backend application at port 8080 within the EKS cluster.

```
GNU nano 5.8 flask-deployment.yaml
spec:
  containers:
    - name: nextwork-flask-backend
      image: 471112976395.dkr.ecr.us-east-1.amazonaws.com/nextwo>
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```




Backend Deployment!

To deploy our backend application, we applied the manifest files that we created. Applying a manifest file means giving Kubernetes the templates for creating the Deployment and Services resources to deploy + expose our application.

kubectl

kubectl is the command line tool for managing resources inside the cluster. We need this tool to deploy the application by creating the Deployment/Service resources. We can't use eksctl for the job because that tool is for creating the cluster.

```
[ec2-user@ip-172-31-29-112 manifests]$ kubectl apply -f flask-deployment.yaml
kubectl apply -f flask-service.yaml
deployment.apps/nextwork-flask-backend created
service/nextwork-flask-backend created
[ec2-user@ip-172-31-29-112 manifests]$
```



Verifying Deployment

Our extension for this project is to use the EKS console to verify our deployment. We had to set up IAM access policies because we don't have access to see our cluster's nodes by default. We set up access by running ``create iamidentitymapping``.

Once we accessed our cluster's nodes, we discovered pods running inside each node. Pods are the smallest deployable unit and represent bundles of containers. Containers in a pod share networking and storage resources, making the app run efficiently.

The EKS console shows you the events for each pod, where we could see our container image getting pulled and used to create a new container running the backend application. This validated that the deployment was a success, the backend is running!

Events (5)				
Type	Reason	Event time	From	Message
✔ Normal	Scheduled	20 minutes ago	default-scheduler	Successfully assigned default/nextwork-flask-backend-67f9cdbb65-kjcmk to ip-1
✔ Normal	Pulling	20 minutes ago	kubelet	Pulling image "471112976395.dkr.ecr.us-east-1.amazonaws.com/nextwork-flask-
✔ Normal	Pulled	20 minutes ago	kubelet	Successfully pulled image "471112976395.dkr.ecr.us-east-1.amazonaws.com/nex
✔ Normal	Created	20 minutes ago	kubelet	Created container nextwork-flask-backend
✔ Normal	Started	20 minutes ago	kubelet	Started container nextwork-flask-backend



NextWork.org

Everyone should be in a job they love.

Check out nextwork.org for
more projects

